UNINA DELIVERY

REALIZZATO DA

FEDERICA FORTINO
ELISA FERRARA
FRANCESCO MATTIA GUZZETTA

Università degli Studi di Napoli Federico II

Documentazione database
Sistema di gestione della logistica delle spedizioni di merci

Federica Fortino N86004648

Elisa Ferrara N86004148

Francesco Mattia Guzzetta N86004738

Anno Accademico 2023–2024

Indice

1 Progettazione concettuale **3**
1.1 Analisi della richiesta . . . 3
1.2 Diagramma ER . . . 4
1.3 Diagramma delle classi UML . . . 5

2 Ristrutturazione del modello concettuale **6**
2.1 Analisi delle ridondanze . . . 6
2.2 Eliminazione degli attributi multi-valore . . . 7
2.3 Eliminazione degli attributi strutturati . . . 7
2.4 Analisi delle generalizzazioni . . . 7
2.5 Identificazione delle chiavi primarie . . . 7
2.6 Modello UML ristrutturato . . . 8

3 Dizionari **9**
3.1 Dizionario delle entità . . . 9
3.2 Dizionario delle associazioni . . . 11
3.3 Dizionario dei vincoli . . . 13
3.3.1 Vincoli intra-relazionali . . . 13
3.3.2 Vincoli inter-relazionali . . . 15

4 Schema logico **16**
4.1 Mapping associazioni . . . 16
4.2 Modello completo . . . 17

5 Schema fisico **19**
5.1 Creazione del database . . . 19
5.2 Struttura SQL delle tabelle . . . 19
5.3 Trigger e trigger function . . . 25
5.4 Funzioni . . . 30
5.5 Procedure . . . 34

6 Implementazione in Java della classe DAO **38**
6.1 Entità ordine . . . 38
6.2 Ordine Dao . . . 39

1 Progettazione concettuale

1.1 Analisi della richiesta

Il sistema UninaDelivery offre agli operatori la possibilità di pianificare, organizzare e monitorare le spedizioni in maniera efficiente. L'operatore può gestire le spedizioni in base a diversi parametri: la disponibilità dei prodotti all'interno del magazzino, il loro peso, la possibilità d'impiego dei mezzi di trasporto e dei corrieri. Le spedizioni possono essere programmate dal cliente che può scegliere, in base alle proprie esigenze, una frequenza per la spedizione del prodotto desiderato.

Ogni spedizione, distinta tramite un numero identificativo, riceve le informazioni riguardo la destinazione del pacco e la società che si occupa del trasporto. Lo stato della spedizione viene continuamente aggiornato in base alla localizzazione del pacco. Salviamo inoltre sia la data di spedizione tramite *data_sped*, sia la data prevista di arrivo tramite *arrivo_prev*.

La spedizione è connessa a un ordine, in quanto si occupa di portare a destinazione gli articoli in esso contenuti. L'ordine è contraddistinto da un numero identificativo e viene completato in una determinata data, *data_ordine*, tramite il pagamento effettuato da un cliente.
Il cliente può scegliere fra tre metodi di pagamento:

- Carta di credito
- Bonifico
- Paypal

Una volta che l'esito del pagamento risulta positivo, l'ordine viene confermato per la spedizione.

Per ogni cliente vengono salvati i dati necessari per eseguire l'ordine, tra cui una email identificativa, un numero di telefono e dati aggiuntivi come il nome, il cognome e la data di nascita. Inoltre, il cliente può scegliere di implementare una periodicità di spedizione di uno o più articoli dell'ordine tramite una programmazione settimanale, mensile o trimestrale. Il cliente ha anche la possibilità di decidere un momento della giornata in cui ricevere il pacco, mediante l'orario.

Per quanto riguarda i prodotti in vendita, salviamo i seguenti attributi: le dimensioni, il peso, il nome e il prezzo. In aggiunta, i prodotti sono contenuti in un unico magazzino e la loro disponibilità viene controllata regolarmente da un operatore. Ogni prodotto dell'ordine viene descritto tramite la classe *dettaglio_ordine* che ne specifica la quantità selezionata.

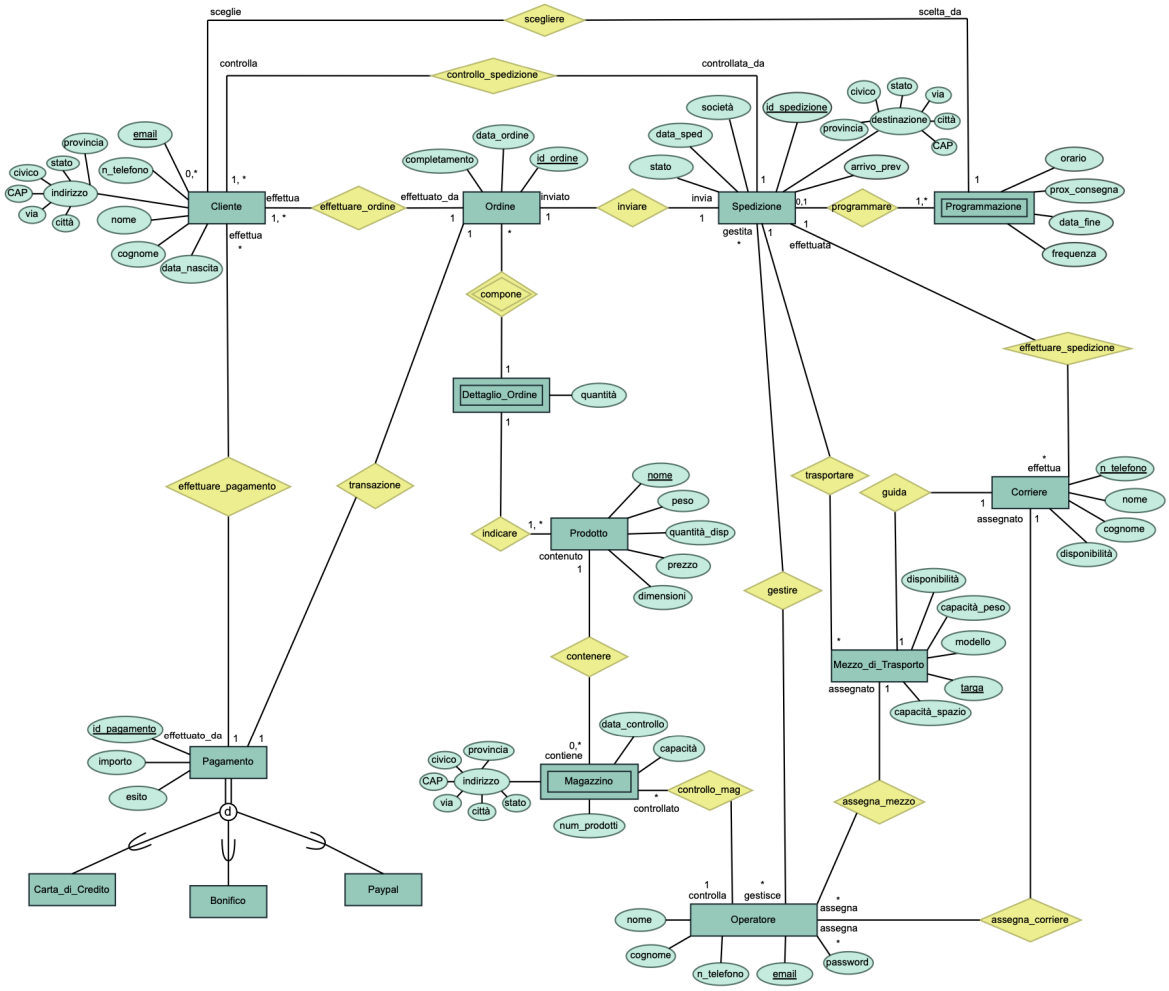
L'operatore è una figura chiave in questo sistema di logistica delle spedizioni, in quanto si occupa dell'assegnazione di un mezzo di trasporto e di un corriere, oltre a dedicarsi alla gestione del magazzino e all'organizzazione della spedizione. Per accedere alla piattaforma, l'operatore usufruisce delle credenziali d'accesso (*email* e *password*).

I mezzi di trasporto sono distinti tramite una targa; ognuno di essi può trasportare un certo peso (*capacità_peso*) e ha uno spazio limitato (*capacità_spazio*). La disponibilità del mezzo è costantemente aggiornata per assicurare l'efficienza della spedizione.

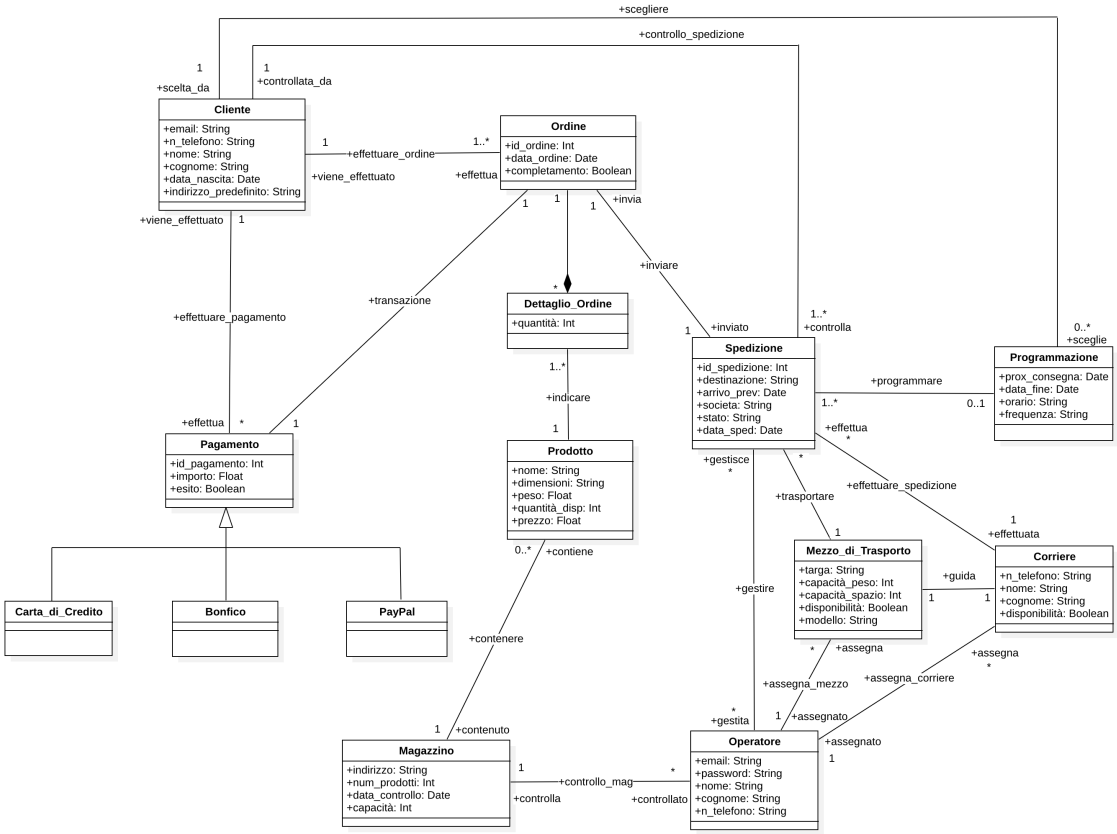
Al sistema partecipano anche i corrieri, dei quali salviamo i dati anagrafici fondamentali (nome, cognome) e il numero di telefono. Essendo registrati nel sistema, la loro disponibilità viene continuamente controllata per essere affidati alle spedizioni.

In conclusione, UninaDelivery rappresenta una piattaforma utile ed efficace, che facilita la gestione delle spedizioni.

1.2 Diagramma ER



1.3 Diagramma delle classi UML



2 Ristrutturazione del modello concettuale

2.1 Analisi delle ridondanze

Sono presenti molteplici relazioni ridondanti. Tra queste troviamo:

- la relazione *controllo_spedizione* è ridondante perché calcolabile tramite la composizione delle relazioni tra Cliente, Ordine e Spedizione.
- la relazione *guida* è ridondante perché calcolabile tramite la composizione delle relazioni tra Mezzo_di_Trasporto, Corriere e Operatore.

Abbiamo inoltre modificato all'interno di alcune classi gli attributi che presentavano la stessa denominazione.

- *email* → *email_cliente* in **Cliente**
- *n_telefono* → *n_telefono_cliente* in **Cliente**
- *nome* → *nome_prodotto* in **Prodotto**
- *email* → *email_operatore* in **Operatore**
- *nome* → *nome_operatore* in **Operatore**
- *cognome* → *cognome_operatore* in **Operatore**
- *n_telefono* → *n_telefono_operatore* in **Operatore**
- *nome* → *nome_corriere* in **Corriere**
- *cognome* → *cognome_corriere* in **Corriere**
- *n_telefono* → *n_telefono_corriere* in **Corriere**

2.2 Eliminazione degli attributi multi-valore

Nel modello UML non sono presenti attributi multi-valore.

2.3 Eliminazione degli attributi strutturati

La prima versione del modello concettuale UML presenta gli attributi strutturati *indirizzo* in **Cliente**, *destinazione* in **Spedizione** e *indirizzo* in **Magazzino**: questi tre attributi sono formati da più parti, ossia la via, il civico, il CAP, la città, la provincia e lo Stato. Per rimuovere questi attributi strutturati nella versione ristrutturata, abbiamo scelto di indicarli con il tipo String.

2.4 Analisi delle generalizzazioni

Nel modello UML non ristrutturato è presente un'unica generalizzazione, che ha come classe padre **Pagamento** e come classi figlie **Carta_di_Credito**, **Bonifico**, **PayPal**. Osservando la generalizzazione, notiamo che questa risulta essere totale e disgiunta. Dunque, accorpiamo le classi specializzate (figlie) all'interno della classe generale (padre); vediamo i motivi di questa scelta:

1. Le classi specializzate non contengono attributi, quindi è molto più semplice gestire una sola classe che le contenga tutte.
2. Riduzione del numero di classi.

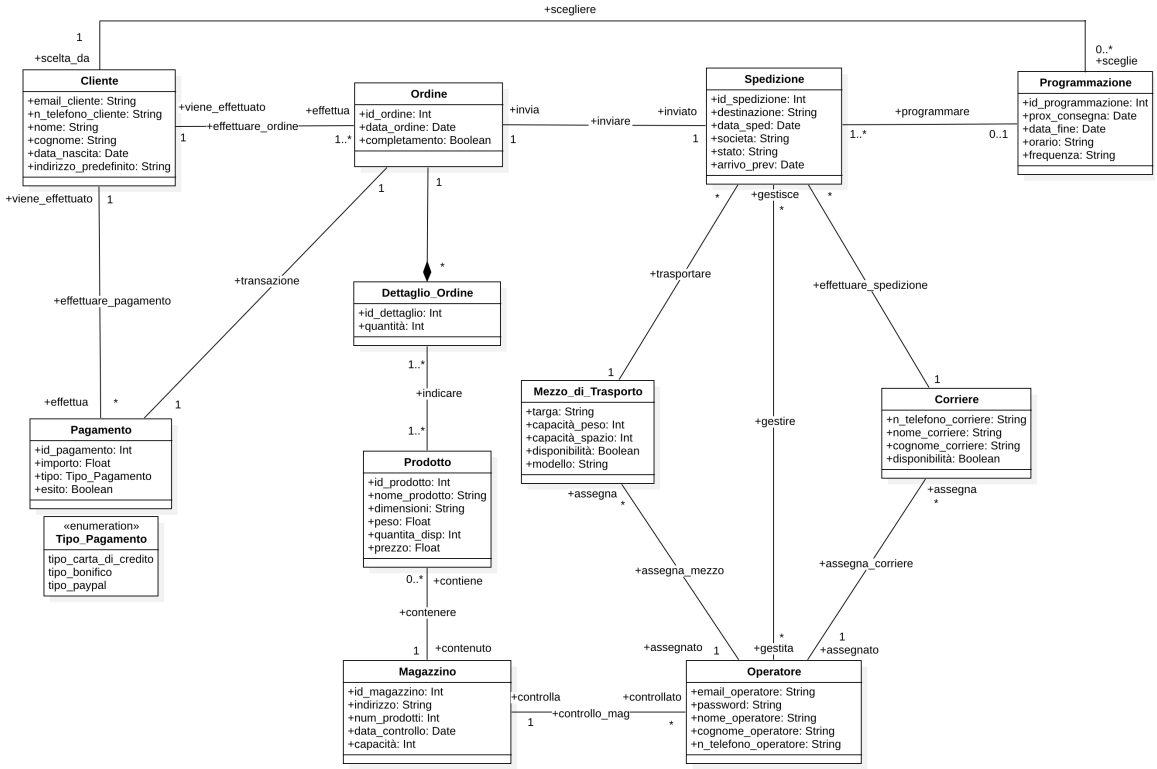
Per via di questa scelta, abbiamo aggiunto un attributo discriminante all'interno della classe **Pagamento**, in modo tale da distinguere i diversi tipi.

2.5 Identificazione delle chiavi primarie

Per la maggior parte delle classi presenti nel modello, abbiamo scelto di inserire una chiave surrogata (avente prefisso *id_*) in modo tale da evitare l'uso di chiavi primarie composte all'interno di una stessa classe. Per le restanti classi abbiamo invece scelto una chiave primaria tra quelle candidate:

- **Cliente** è identificato dall'attributo *email_cliente*.
- **Operatore** è identificato dall'attributo *email_operatore*.
- **Mezzo_di_Trasporto** è identificato dall'attributo *targa*, una stringa di 7 caratteri.
- **Corriere** è identificato dall'attributo *n_telefono_corriere*, una stringa di 10 caratteri.

2.6 Modello UML ristrutturato



3 Dizionari

3.1 Dizionario delle entità

Classe	Descrizione	Attributi
Cliente	Soggetto coinvolto nell'acquisto dei prodotti tramite un ordine.	**Email_cliente** (string): Chiave primaria, identifica univocamente ciascuna istanza del Cliente. **N_telefono_cliente** (string): Numero di telefono associato al Cliente. **Nome** (string): Nome associato al Cliente. **Cognome** (string): Cognome associato al Cliente. **Data_nascita** (date): Data di nascita del Cliente. **Indirizzo** (string): Indirizzo primario scelto per la consegna.
Pagamento	Descrittore dei dati e dei metodi di pagamento, necessari per concludere l'ordine.	**Id_pagamento** (int): Chiave primaria, identifica univocamente ciascuna istanza di Pagamento. **Importo** (float): Cifra necessaria a concludere il pagamento. **Tipo** (Tipo_Pagamento): Indica il tipo di pagamento. **Esito** (boolean): Indica se il pagamento è stato completato oppure no.
Ordine	Descrittore dei dati dell'ordine effettuato dal cliente.	**Id_ordine** (int): Chiave primaria, identifica univocamente ciascuna istanza dell'ordine. **Data_ordine** (date): Data in cui è stato effettuato l'ordine. **Completamento** (boolean): Indica se l'ordine è stato completato o meno.
Dettaglio_Ordine	Descrittore dei dettagli di ogni articolo presente nell'ordine.	**Id_dettaglio** (int): Chiave primaria, identifica univocamente le istanze di Dettaglio_Ordine. **Quantità** (int): Quantità di prodotto selezionato all'interno dell'ordine.
Prodotto	Descrittore di un articolo destinato alla vendita.	**Id_prodotto** (int): Chiave primaria, identifica univocamente le istanze di prodotto. **Nome_prodotto** (string): Nome associato al prodotto. **Dimensioni** (float): Dimensioni dell'articolo. **Peso** (float): Peso dell'articolo. **Quantità_disp** (int): Quantità del prodotto disponibile. **Prezzo** (float): Costo associato ad un'unità di Prodotto.

Magazzino	Descrittore del luogo in cui è contenuta la merce destinata alla vendita.	**Id_magazzino** (int): Chiave primaria, identifica univocamente le istanze di Magazzino. **Indirizzo (string)**: Ubicazione del Magazzino per il deposito dei prodotti. **Num_prodotti** (int): Numero dei prodotti presenti nel Magazzino. **Data_controllo** (date): Data in cui è stato effettuato il controllo della disponibilità nel Magazzino. **Capacità** (int): Quantità massima dei prodotti nel Magazzino.
Operatore	Soggetto che si occupa dell'organizzazione e della pianificazione delle spedizioni.	**Email_operatore** (string): Chiave primaria, identifica univocamente le istanze dell'Operatore. **Password** (string): Codice utilizzato dall'operatore per effettuare il login. **Nome_operatore** (string): Nome associato all'Operatore. **Cognome** (string): Cognome associato all'Operatore. **N_telefono_operatore** (string): Numero di Telefono associato all'Operatore.
Mezzo_di_Trasporto	Descrittore del mezzo di trasporto utilizzabile per effettuare la spedizione.	**Targa** (string): Chiave primaria, identifica univocamente ciascuna istanza di Mezzo_di_Trasporto. **Capacità_peso** (int): Quantità massima relativa al peso trasportabile dal veicolo. **Capacità_spazio** (int): Quantità massima relativa allo spazio disponibile nel veicolo in termini di volume. **Disponibilità** (boolean): Possibilità di usufruire del veicolo. **Modello** (string): Informazioni relative al modello del veicolo.
Corriere	Soggetto che si occupa di condurre fisicamente la spedizione.	**N_telefono_corriere** (string): Chiave primaria, identifica univocamente ciascuna istanza di Corriere. **Nome** (string): Nome associato al Corriere. **Cognome** (string): Cognome associato al Corriere. **Disponibilità** (boolean): possibilità di impiegare un Corriere.
Spedizione	Descrittore del processo di trasporto dei prodotti dal magazzino all'acquirente.	**Id_spedizione** (int): Chiave primaria, identifica univocamente ciascuna istanza di Spedizione. **Destinazione** (string): Luogo di arrivo e di fine della Spedizione. **Arrivo_prev** (date): Data in cui l'ordine viene spedito. **Società** (string): Società che si prende cura della Spedizione. **Stato** (string): Stato corrente della Spedizione.
Programmazione	Descrittore della pianificazione della periodicità di un acquisto.	**Id_programmazione** (int): chiave primaria, identifica univocamente ciascuna istanza di Programmazione. **Prox_consegna** (date): Data in cui la Programmazione ha inizio. **Data_fine** (date): Data in cui la Programmazione finisce. **Orario** (string): Orario in cui è pianificata la consegna dell'ordine. **Frequenza** (string): Intervallo di giorni nella Programmazione tra una consegna e l'altra.

3.2 Dizionario delle associazioni

Associazione	Descrizione	Classi coinvolte
Scegliere	Esprime la scelta del cliente sulla periodicità dell'ordine.	**Cliente** [**0..***] *ruolo* (**sceglie**): Indica la scelta che il cliente compie. **Programmazione** [**1**] *ruolo* (**scelta_da**): Indica la scelta compiuta dal cliente.
Effettuare_ordine	Esprime che il cliente effettua l'ordine.	**Cliente** [**1..***] *ruolo* (**effettua**): Indica che il cliente effettua l'ordine. **Ordine** [**1**] *ruolo* (**viene_effettuato**): Indica che l'ordine viene effettuato dal cliente.
Effettuare_pagamento	Esprime che il cliente effettua il pagamento.	**Cliente** [*] *ruolo* (**effettua**): Indica che il cliente effettua il pagamento. **Pagamento** [**1**] *ruolo* (**viene_effettuato**): Indica che il pagamento viene effettuato dal cliente.
Transazione	Esprime la relazione di pagamento necessario a concludere l'ordine.	**Pagamento** [**1**] *ruolo* (**conclude**): Indica il completamento dell'ordine una volta effettuato il pagamento. **Ordine** [**1**] *ruolo* (**necessita**): Indica il bisogno di un pagamento concluso per completare l'ordine.
Indicare	Esprime l'implementazione di un dettaglio ordine che indica uno specifico prodotto.	**Dettaglio_ordine** [**1..***] *ruolo* (**indica**): Indica che il dettaglio ordine si riferisce ad un prodotto specifico. **Prodotto** [**1**] *ruolo* (**indicato**): Indica che il prodotto viene descritto nel dettaglio ordine.
Contenere	Esprime la presenza del prodotto nel magazzino.	**Prodotto** [**1**] *ruolo* (**contenuto**): Indica che il prodotto si trova nel magazzino. **Magazzino** [**0..***] *ruolo* (**contiene**): Indica che il magazzino contiene il prodotto.
Contenere	Esprime la presenza del prodotto nel magazzino.	**Prodotto** [**1**] *ruolo* (**contenuto**): Indica che il prodotto si trova nel magazzino. **Magazzino** [**0..***] *ruolo* (**contiene**): Indica che il magazzino contiene il prodotto.
Inviare	Esprime l'invio dell'ordine tramite la spedizione.	**Ordine** [**1**] *ruolo* (**inviato**): Indica che l'ordine è inviato tramite la spedizione. **Spedizione** [**1**] *ruolo* (**invia**): Indica che la spedizione si prende carico dell'ordine.
Programmare	Esprime la pianificazione della periodicità della spedizione.	**Spedizione** [**0..1**] *ruolo* (**pianificata**): Indica la pianificazione della spedizione tramite la programmazione. **Programmazione** [**1..***] *ruolo* (**pianifica**): Indica il ruolo della programmazione nella pianificazione delle spedizioni.
Gestire	Esprime una relazione tra l'operatore e la gestione della spedizione.	**Operatore** [*] *ruolo* (**gestisce**): Indica che l'operatore si occupa della gestione. **Spedizione** [*] *ruolo* (**gestita**): Indica che la spedizione viene gestita dall'operatore.

Controllo_mag	Esprime il controllo della disponibilità nel magazzino da parte dell'operatore.	**Operatore** [**1**] *ruolo* (**controlla**): Indica il ruolo dell'operatore nel controllare la disponibilità dei prodotti nel magazzino. **Magazzino** [*****] *ruolo* (**controllato**): Indica che il controllo della disponibilità dei prodotti nel magazzino viene gestito dall'operatore.
Assegna_mezzo	Esprime l'assegnazione di un mezzo di trasporto dall'operatore per la spedizione.	**Operatore** [*****] *ruolo* (**assegna**): Indica che l'operatore assegna un veicolo. **Mezzo_di_Trasporto** [**1**] *ruolo* (**assegnato**): Indica che il mezzo viene assegnato dall'operatore.
Assegna_corriere	Esprime l'assegnazione di un corriere dall'operatore per la spedizione.	**Operatore** [*****] *ruolo* (**assegna**): Indica che l'operatore sceglie un corriere. **Corriere** [**1**] *ruolo* (**assegnato**): Indica che il corriere viene assegnato da un operatore.
Trasportare	Esprime il trasporto degli articoli tramite un mezzo di trasporto.	**Spedizione** [*****] *ruolo* (**trasporta**): Indica che il mezzo di trasporto trasporta la spedizione . **Mezzo_di_Trasporto** [**1**] *ruolo* (**viene_trasportata**): Indica che la spedizione viene trasportasta dal mezzo di trasporto.
Effettuare_spedizione	Esprime il trasporto degli articoli tramite un mezzo di trasporto.	**Spedizione** [*****] *ruolo* (**viene_effettuata**): Indica che il corriere trasporta la spedizione. **Corriere** [**1**] *ruolo* (**effettua**):Indica che la spedizione viene trasportata dal corriere.

3.3 Dizionario dei vincoli

3.3.1 Vincoli intra-relazionali

Cliente

Nome	Descrizione
PKEmailCliente	L'email di un Cliente deve essere univoca e non nulla.
uniqueNumTelCliente	Il numero di telefono di un Cliente deve essere univoco.

Pagamento

Nome	Descrizione
PKIdPagamento	L'id_pagamento deve essere univoco e non nullo.
minValoreImporto	L'importo deve avere valore maggiore di zero.
constraintTypePagamento	Il tipo di pagamento deve essere uno di quelli elencati nell'enumeration.
successRulePagamento	Il pagamento è andato a buon fine solo se esito ha valore true.

Ordine

Nome	Descrizione
PKIdOrdine	L'id_ordine deve essere univoco e non nullo.
successRuleOrdine	L'ordine è andato a buon fine solo se completamento ha valore true.

Dettaglio_Ordine

Nome	Descrizione
PKIdDettaglio	L'id_dettaglio deve essere univoco e non nullo.
minValoreQuantità	La quantità deve avere valore maggiore di zero.

Magazzino

Nome	Descrizione
PKIdMagazzino	L'id_magazzino deve essere univoco e non nullo.
constraintCapacità	Num_prodotti deve essere minore o uguale alla capacità.

Prodotto

Nome	Descrizione
PKEmailOperatore	L'email di un Operatore deve essere univoca e non nulla.
uniqueNTelOperatore	N_telefono_operatore deve essere univoco.

Spedizione

Nome	Descrizione
PKIdSpedizione	L'id_spedizione deve essere univoco e non nullo.
checkStato	Lo stato di Spedizione può assumere le seguenti forme: "ordinato", "spedito", "consegnato".

Programmazione

Nome	Descrizione
PKIdProgrammazione	L'id_programmazione deve essere univoco e non nullo.
checkFrequenza	La frequenza deve assumere una delle seguenti forme: "Settimanale", "Mensile", "Trimestrale".
validDateRange	Prox_consegna dev'essere precedente o coincidere con Data_fine.
checkOrario	Orario può assumere una delle seguenti forme: "Mattina", "Pomeriggio".

Mezzo_di_Trasporto

Nome	Descrizione
PKTargaMezzo	La targa di un mezzo di trasporto deve essere univoca e non nulla

Corriere

Nome	Descrizione
PKNTelefonoCorriere	N_telefono_corriere deve essere univoco e non nullo.

3.3.2 Vincoli inter-relazionali

Nome	Descrizione
isValidEffettuarePagamento	Un Pagamento deve essere necessariamente associato ad un Cliente e la relazione esiste solo se esiste Id_pagamento.
isValidEffettuareOrdine	Un Ordine deve essere necessariamente associato ad un Cliente e la relazione esiste solo se esiste Id_Ordine.
amountPagamento	L'importo del Pagamento deve corrispondere al prezzo_tot dell'Ordine.
isValidTransazione	Se Id_Ordine è NULL, l'entità Pagamento non esiste
existDettaglioOrdine	L'esistenza della classe Dettaglio_Ordine dipende da Ordine.
isValidInviare	La relazione tra Spedizione e Ordine esiste solo se Ordine ha completamento True.
dataOrdineBeforeDataSped	Nella relazione tra Spedizione e Ordine, data_ordine deve essere una data precedente a data_sped.
isValidProgrammare	Una Programmazione deve essere necessariamente associata ad una Spedizione e la relazione esiste solo se esiste un id_spedizione.
constraintArrivo	Se esiste id_programmazione, allora arrivo_prev di Spedizione coincide con prox_consegna.
isValidContenere	Se Magazzino non esiste, non può esistere la classe Prodotto.
isValidIndicare	Un Dettaglio_Ordine deve essere necessariamente associato ad un Prodotto.
isValidControlloMag	La relazione tra operatore e magazzino esiste solo se al magazzino c'è un operatore associato
isValidAssegnaMezzo	Ogni mezzo di trasporto viene scelto da un operatore associato.
isValidAssegnaCorriere	Ogni corriere viene scelto da un operatore associato.
isValidScegliere	Ogni programmazione ha un cliente associato.
isValidTrasportare	Ogni spedizione ha un mezzo di trasporto associato.
isValidEffettuareSpedizione	Ogni spedizione ha un corriere associato.

4 Schema logico

4.1 Mapping associazioni

Associazioni 1-1

- Ordine-Pagamento: inserimento della chiave primaria id_pagamento di Pagamento nella classe Ordine come chiave esterna
- Spedizione-Ordine: inserimento della chiave primaria id_ordine di Ordine nella classe Spedizione come chiave esterna

Associazioni 1-N

- Cliente-Pagamento: migriamo la chiave primaria della classe forte della relazione Cliente, email_cliente, nella classe debole Pagamento
- Cliente-Programmazione: migriamo la chiave primaria della classe forte della relazione Cliente, email_cliente, nella classe debole Programmazione
- Cliente-Ordine: migriamo la chiave primaria della classe forte della relazione Cliente, email_cliente, nella classe debole Ordine
- Ordine-Dettaglio_Ordine: migriamo la chiave primaria della classe forte della relazione Ordine, id_ordine, nella classe debole Dettaglio_Ordine
- Dettaglio_Ordine-Prodotto: migriamo la chiave primaria della classe forte della relazione Prodotto, id_prodotto, nella classe debole Dettaglio_Ordine
- Magazzino-Prodotto: migriamo la chiave primaria della classe forte della relazione Magazzino, id_magazzino, nella classe debole Prodotto
- Magazzino-Operatore: migriamo la chiave primaria della classe forte della relazione Magazzino, id_magazzino, nella classe debole Operatore
- Operatore-Mezzo_di_Trasporto: migriamo la chiave primaria della classe forte della relazione Operatore, email_operatore, nella classe debole Mezzo_di_Trasporto

- Operatore-Corriere: migriamo la chiave primaria della classe forte della relazione Operatore, email_operatore, nella classe debole Corriere

- Programmazione-Spedizione: migriamo la chiave primaria della classe forte della relazione Programmazione, id_programmazione, nella classe debole Spedizione

Associazioni N-N

Andiamo a creare le tabelle ponte per le relazioni molti a molti.
Spedizione-Operatore:

Gestire
id_spedizione, email_operatore

4.2 Modello completo

In rosso abbiamo le PK e in verde le FK.

Cliente
email_cliente, n_telefono_cliente, nome, cognome, data_nascita, indirizzo_predefinito

Pagamento
id_pagamento, importo, tipo, esito, email_cliente

Ordine
id_ordine, data_ordine, completamento, id_pagamento , email_cliente

Dettaglio_Ordine
id_dettaglio, quantità, id_ordine, id_prodotto

Magazzino
id_magazzino, indirizzo, num_prodotti, data_controllo, capacità

Prodotto
id_prodotto, nome_prodotto, dimensioni, peso, quantità_disp, prezzo, id_magazzino

Operatore
email_operatore, password, nome_operatore, cognome_operatore, n_telefono_operatore, id_magazzino

Mezzo_di_Trasporto
targa, capacità_peso, capacità_spazio, disponibilità, modello, email_operatore

Corriere
n_telefono_corriere, nome_corriere, cognome_corriere, disponibilità, email_operatore

Programmazione
id_programmazione, prox_consegna, data_fine, orario, frequenza, email_cliente

Spedizione
id_spedizione, destinazione, data_sped, società, stato, arrivo_prev, id_ordine,id_programmazione

Gestire
id_spedizione, email_operatore

5 Schema fisico

5.1 Creazione del database

```sql
CREATE DATABASE UninaDelivery;
```

5.2 Struttura SQL delle tabelle

Tabella Cliente

```sql
CREATE TABLE Cliente
(
    email_cliente character varying(100) NOT NULL,
    n_telefono_cliente character(10),
    nome character varying(30)  NOT NULL,
    cognome character varying(30)  NOT NULL,
    data_nascita date NOT NULL,
    indirizzo_predefinito character varying(200) NOT NULL,
    CONSTRAINT "PKEmailCliente" PRIMARY KEY (email_cliente),
    CONSTRAINT "uniqueNumTelCliente" UNIQUE (n_telefono_cliente)
)
```

Tabella Pagamento

```sql
CREATE TABLE Pagamento
(
    id_pagamento integer NOT NULL,
    importo double precision NOT NULL,
    tipo tipo_pagamento NOT NULL,
    esito boolean,
    email_cliente character varying(100)  NOT NULL,
    CONSTRAINT "PKIdPagamento" PRIMARY KEY (id_pagamento),
    CONSTRAINT "isValidEffettuarePagamento" FOREIGN KEY (email_cliente)
        REFERENCES Cliente (email_cliente)
```

```
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT "minValoreImporto" CHECK (importo > 0),
    CONSTRAINT "constraintTypePagamento" CHECK (tipo=('tipo_carta_di_credito',
    'tipo_paypal','tipo_bonifico')),
    CONSTRAINT "successRulePagamento" CHECK (esito = true)
)
```

Tabella Ordine

```
CREATE TABLE Ordine
(
    id_ordine integer NOT NULL,
    data_ordine date NOT NULL,
    completamento boolean,
    id_pagamento integer,
    email_cliente character varying(100)  NOT NULL,
    CONSTRAINT "PKIdOrdine" PRIMARY KEY (id_ordine),
    CONSTRAINT "isValidEffettuareOrdine" FOREIGN KEY (email_cliente)
        REFERENCES Cliente (email_cliente)
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT "isValidTransazione" FOREIGN KEY (id_pagamento)
        REFERENCES Pagamento (id_pagamento)
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT "successRuleOrdine" CHECK (completamento = true)
)
```

Tabella Dettaglio_Ordine

```
CREATE TABLE Dettaglio_Ordine
(
    id_dettaglio integer NOT NULL,
    quantita integer NOT NULL,
    id_ordine integer NOT NULL,
    id_prodotto integer NOT NULL,
    CONSTRAINT "PKIdDettaglio" PRIMARY KEY (id_dettaglio),
    CONSTRAINT "existDettaglioOrdine" FOREIGN KEY (id_ordine)
        REFERENCES Ordine (id_ordine)
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT "isValidIndicare" FOREIGN KEY (id_prodotto)
        REFERENCES Prodotto (id_prodotto)
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT "minValoreQuantità" CHECK (quantita > 0)
)
```

Tabella Magazzino

```sql
CREATE TABLE Magazzino
(
    id_magazzino integer NOT NULL,
    indirizzo character varying(200) NOT NULL,
    num_prodotti integer NOT NULL,
    data_controllo date NOT NULL,
    capacita integer NOT NULL,
    CONSTRAINT "PKIdMagazzino" PRIMARY KEY (id_magazzino),
    CONSTRAINT "constraintCapacita" CHECK (num_prodotti <= capacita)
)
```

Tabella Prodotto

```sql
CREATE TABLE Prodotto
(
    id_prodotto integer NOT NULL,
    nome_prodotto character varying(300) NOT NULL,
    dimensioni character varying(40) NOT NULL,
    peso double precision NOT NULL,
    quantita_disp integer NOT NULL,
    prezzo double precision NOT NULL,
    id_magazzino integer NOT NULL,
    CONSTRAINT "PKIdProdotto" PRIMARY KEY (id_prodotto),
    CONSTRAINT "isValidContenere" FOREIGN KEY (id_magazzino)
        REFERENCES Magazzino (id_magazzino)
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT "minValorePrezzo" CHECK (prezzo > 0)
)
```

Tabella Operatore

```
CREATE TABLE Operatore
(
    email_operatore character varying(100) NOT NULL,
    password character varying(32) NOT NULL,
    nome_operatore character varying(30) NOT NULL,
    cognome_operatore character varying(30) NOT NULL,
    n_telefono_operatore character(10),
    id_magazzino integer NOT NULL,
    CONSTRAINT "PKEmailOperatore" PRIMARY KEY (email_operatore),
    CONSTRAINT "uniqueNTelOperatore" UNIQUE (n_telefono_operatore),
    CONSTRAINT "isValidControlloMag" FOREIGN KEY (id_magazzino)
        REFERENCES Magazzino (id_magazzino)
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
)
```

Tabella Mezzo_Di_Trasporto

```
CREATE TABLE Mezzo_di_Trasporto
(
    targa character varying(7)  NOT NULL,
    capacita_peso integer NOT NULL,
    capacita_spazio integer NOT NULL,
    disponibilita boolean,
    modello character varying(300) NOT NULL,
    email_operatore character varying(100)  NOT NULL,
    CONSTRAINT "PKTarga" PRIMARY KEY (targa),
    CONSTRAINT "isValidAssegnaMezzo" FOREIGN KEY (email_operatore)
        REFERENCES Operatore (email_operatore)
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
)
```

Tabella Corriere

```sql
CREATE TABLE Corriere
(
    n_telefono_corriere character(10) NOT NULL,
    nome_corriere character varying(30)  NOT NULL,
    cognome_corriere character varying(30)  NOT NULL,
    disponibilita boolean,
    email_operatore character varying(100) NOT NULL,
    CONSTRAINT "PKNTelCorriere" PRIMARY KEY (n_telefono_corriere),
    CONSTRAINT "isValidAssegnaCorriere" FOREIGN KEY (email_operatore)
        REFERENCES Operatore (email_operatore)
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
)
```

Tabella Programmazione

```sql
CREATE TABLE Programmazione
(
    id_programmazione integer NOT NULL,
    prox_consegna date,
    data_fine date NOT NULL,
    orario character varying(20)  DEFAULT 'mattina',
    frequenza character varying(20),
    email_cliente character varying(100) NOT NULL,
    CONSTRAINT "PKIdProgrammazione" PRIMARY KEY (id_programmazione),
    CONSTRAINT "isValidScegliere" FOREIGN KEY (email_cliente)
        REFERENCES Cliente (email_cliente)
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT "validDateRange" CHECK (prox_consegna <= data_fine),
    CONSTRAINT "checkOrario" CHECK (orario = ('mattina', 'pomeriggio')),
    CONSTRAINT "checkFrequenza" CHECK (frequenza = ( 'settimanale', 'mensile', 'trimestrale'))
)
```

Tabella Spedizione

```sql
CREATE TABLE Spedizione
(
    id_spedizione integer NOT NULL,
    destinazione character varying(200) NOT NULL,
    data_sped date,
    societa character varying(100) NOT NULL,
    stato character varying(20) DEFAULT 'ordinato',
    arrivo_prev date NOT NULL,
    id_ordine integer NOT NULL,
    id_programmazione integer,
    targa character(7),
    n_telefono_corriere character(10),
    CONSTRAINT "PKIdSpedizione" PRIMARY KEY (id_spedizione),
    CONSTRAINT "isValidEffettuareSpedizione" FOREIGN KEY (n_telefono_corriere)
        REFERENCES Corriere (n_telefono_corriere)
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT "isValidInviare" FOREIGN KEY (id_ordine)
        REFERENCES Ordine (id_ordine)
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT "isValidProgrammare" FOREIGN KEY (id_programmazione)
        REFERENCES Programmazione (id_programmazione)
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT "isValidTrasportare" FOREIGN KEY (targa)
        REFERENCES Mezzo_di_Trasporto (targa)
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT "checkStato" CHECK (stato = ('ordinato', 'spedito', 'consegnato')),
    CONSTRAINT constraintarrivo CHECK (checkconstraintarrivo(id_programmazione, id_spedizione))
)

```

5.3 Trigger e trigger function

UpdateCompletamentoOrdine

Un Ordine ha completamento True se Pagamento ha esito True.
Iniziamo con la creazione di una funzione UpdateCompletamentoOrdine(): quando l'esito del pagamento è TRUE viene fatto l'UPDATE del completamento dell'Ordine. Questa funzione verrà eseguita all'interno del Trigger.

```
CREATE OR REPLACE FUNCTION UpdateCompletamentoOrdine()
RETURNS TRIGGER
AS $$
DECLARE
BEGIN
  IF NEW.Esito = TRUE THEN
    UPDATE Ordine
    SET completamento = TRUE
    WHERE id_pagamento = NEW.id_pagamento;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER Trigger_UpdateCompletamentoOrdine
AFTER UPDATE ON Pagamento
FOR EACH ROW
EXECUTE FUNCTION UpdateCompletamentoOrdine();
```

isValidAssegnaMezzo

L'operatore non può scegliere un mezzo di trasporto se questo non è disponibile.
Iniziamo con la creazione di una funzione isValidAssegnaMezzo(): se la disponibilità del mezzo di trasporto è FALSE allora viene inviato un messaggio di errore che non permette all'operatore di scegliere quel mezzo ed assegnarlo ad una spedizione. Questa funzione viene poi eseguita nel Trigger associato.

```
CREATE OR REPLACE FUNCTION isValidAssegnaMezzo()
RETURNS TRIGGER
AS $$
BEGIN
        IF NOT EXISTS (SELECT 1
                              FROM Mezzo_di_Trasporto
                              WHERE targa = NEW.targa AND disponibilita = TRUE) THEN
                RAISE EXCEPTION 'Impossibile associare un operatore a un mezzo di trasporto non
                disponibile.';
        END IF;
        RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER Trigger_IsValidAssegnaMezzo
BEFORE UPDATE ON Mezzo_di_Trasporto
```

```
FOR EACH ROW
EXECUTE FUNCTION isValidAssegnaMezzo();
```

isValidAssegnaCorriere

L'operatore non può scegliere un corriere se questo non è disponibile.
Iniziamo con la creazione di una funzione **isValidAssegnaCorriere()**: se la disponibilità del corriere è FALSE allora viene inviato un messaggio di errore che non permette all'operatore di scegliere quel mezzo ed assegnarlo ad una spedizione. Questa funzione viene poi eseguita nel Trigger associato.

```
CREATE OR REPLACE FUNCTION isValidAssegnaCorriere()
RETURNS TRIGGER
AS $$
BEGIN
        IF NOT EXISTS (SELECT 1
                                FROM Corriere
                                WHERE n_telefono_corriere = NEW.n_telefono_corriere AND disponibilita
                                = TRUE) THEN
                RAISE EXCEPTION 'Impossibile associare un operatore a un corriere non disponibile.';
        END IF;
        RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER Trigger_IsValidAssegnaCorriere
BEFORE UPDATE ON Corriere
FOR EACH ROW
EXECUTE FUNCTION isValidAssegnaCorriere();
```

checkQuantitaScelta

La quantità in Dettaglio_Ordine deve essere minore alla quantità di Prodotto.
Creiamo una funzione che, dopo aver ottenuto la quantità disponibile del prodotto va a verificare se la quantità selezionata nel Dettaglio_Ordine supera la quantità disponibile del Prodotto. Questa funzione viene eseguita all'interno del Trigger associato.

```
CREATE OR REPLACE FUNCTION checkQuantitaScelta()
RETURNS TRIGGER
AS $$
DECLARE
                quantita_prodotto INT;
BEGIN
                        SELECT quantita_disp INTO quantita_prodotto
                        FROM Prodotto
                        WHERE id_prodotto = NEW.id_prodotto;

                        IF NEW.quantita > quantita_prodotto THEN
                                RAISE EXCEPTION'La quantità selezionata supera la quantità disponibile
                                del Prodotto.';
                        END IF;
RETURN NEW;
```

```
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER Trigger_CheckQuantitaScelta
BEFORE INSERT ON Dettaglio_Ordine
FOR EACH ROW
EXECUTE FUNCTION checkQuantitaScelta();
```

AssegnaMezzoDiTrasporto

Questo trigger utilizza una funzione che assegna un mezzo di trasporto alla spedizione in base alla sua disponibilità.

```
CREATE OR REPLACE FUNCTION AssegnaMezzoDiTrasporto()
RETURNS TRIGGER
AS $$
DECLARE
targa_mezzo VARCHAR(7);
BEGIN
   SELECT targa INTO targa_mezzo
   FROM Mezzo_di_Trasporto
   WHERE disponibilita = TRUE
   LIMIT 1;

   UPDATE Spedizione
   SET targa_mezzo = NEW.targa_mezzo
   WHERE id_spedizione = NEW.id_spedizione;

   UPDATE Mezzo_di_Trasporto
   SET disponibilita = FALSE
   WHERE targa = targa_mezzo;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER Trigger_AssegnaMezzoDiTrasporto
AFTER INSERT ON Spedizione
FOR EACH ROW
EXECUTE FUNCTION AssegnaMezzoDiTrasporto();
```

AssegnaCorriere

Questo trigger utilizza una funzione che assegna un corriere alla spedizione in base alla sua disponibilità.

```
CREATE OR REPLACE FUNCTION AssegnaCorriere()
RETURNS TRIGGER
AS $$
DECLARE
p_n_telefono_corriere CHAR(10);
BEGIN
   SELECT p_n_telefono_corriere INTO n_telefono_corriere
   FROM Corriere
```

```
    WHERE disponibilita = TRUE
    LIMIT 1;

    UPDATE Spedizione
    SET p_n_telefono_corriere = NEW.n_telefono_corriere
    WHERE id_spedizione = NEW.id_spedizione;

    UPDATE Corriere
    SET disponibilita = FALSE
    WHERE n_telefono:corriere = p_n_telefono_corriere;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER Trigger_AssegnaCorriere
AFTER UPDATE ON Spedizione
FOR EACH ROW
EXECUTE FUNCTION AssegnaCorriere();
```

AssegnaProgrammazione

Ogni volta che un cliente sceglie una Programmazione per il proprio Ordine, questa viene associata alla Spedizione di esso.

```
CREATE OR REPLACE FUNCTION AssegnaProgrammazione()
RETURNS TRIGGER
AS $$
BEGIN
     UPDATE Spedizione
     SET id_programmazione = NEW.id_programmazione
     WHERE id_ordine = NEW.id_ordine AND NEW.id_programmazione IS NOT NULL;
     RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER Trigger_AssegnaProgrammazione
AFTER INSERT ON Ordine
FOR EACH ROW
EXECUTE FUNCTION AssegnaProgrammazione();
```

InsertSpedizioneProg

Ogni volta che prox_consegna viene aggiornato, i dati di spedizione vengono aggiornati.

```
CREATE OR REPLACE FUNCTION assegnaProgrammazione()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.id_programmazione IS NOT NULL THEN
        UPDATE Spedizione
        SET id_programmazione = NEW.id_programmazione
        WHERE id_spedizione = NEW.id_spedizione;
```

```
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_assegnaProgrammazione
AFTER INSERT ON Programmazione
FOR EACH ROW
EXECUTE FUNCTION assegnaProgrammazione();
```

UpdateDataSped

Nel momento in cui viene spedito l'ordine, la data di spedizione viene aggiornata alla data corrente.

```
CREATE OR REPLACE FUNCTION UpdateDataSped()
RETURNS TRIGGER
AS $$
DECLARE
BEGIN
    IF NEW.stato = 'spedito' AND OLD.stato <> 'spedito' THEN
        UPDATE Spedizione
        SET data_sped = CURRENT_DATE
        WHERE id_spedizione = NEW.id_spedizione;
    END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER Trigger_UpdateDataSped
BEFORE UPDATE OF stato ON Spedizione
FOR EACH ROW
EXECUTE FUNCTION UpdateDataSped();
```

beforeinsertupdatepassword

```
CREATE OR REPLACE FUNCTION public.beforeinsertupdatepassword()
    RETURNS trigger
    LANGUAGE 'plpgsql'
AS $BODY$
BEGIN
        NEW.password = MD5Password(NEW.password);
        RETURN NEW;
END
$BODY$;

CREATE TRIGGER Trigger_HashingPassword
BEFORE INSERT OR UPDATE ON Operatore
FOR EACH ROW
EXECUTE FUNCTION beforeInsertUpdatePassword();
```

5.4 Funzioni

ElencoSpedizioniProgrammate

Restituisce un elenco delle spedizioni programmate per un cliente specifico.

```
CREATE OR REPLACE FUNCTION ElencoSpedizioniProgrammate(email_cliente IN VARCHAR(100))
RETURNS SETOF Spedizione
AS $$
DECLARE
    tmp_record Spedizione%ROWTYPE;
    SelezionaSpedizioni CURSOR FOR
        SELECT
            S.id_spedizione,
            S.destinazione,
            S.data_sped,
            S.societa,
            S.stato,
            S.arrivo_prev
        FROM Spedizione AS S
        JOIN Ordine AS O ON S.id_ordine = O.id_ordine
        JOIN Cliente AS C ON O.email_cliente = C.email_cliente
        JOIN Programmazione P ON S.id_programmazione = P.id_programmazione
        WHERE C.email_cliente = email_cliente AND
            P.frequenza IN ('settimanale', 'mensile', 'trimestrale');
BEGIN
    OPEN SelezionaSpedizioni;
    LOOP
        FETCH SelezionaSpedizioni INTO tmp_record;
        EXIT WHEN NOT FOUND;
        RETURN NEXT tmp_record;
    END LOOP;
    CLOSE SelezionaSpedizioni;
    RETURN;
END;
$$ LANGUAGE plpgsql;
```

CostoTotOrdine

Questa funzione restituisce il costo totale di un ordine in base alla somma di tutti i prodotti per il loro prezzo.

```
CREATE OR REPLACE FUNCTION CostoTotOrdine
(p_Id_Ordine IN INTEGER)
RETURNS FLOAT
AS $$
DECLARE
CostoTotale FLOAT;
BEGIN
                        SELECT SUM(DETT.quantita * PR.prezzo) INTO CostoTotale
                        FROM Dettaglio_Ordine AS DETT
                        JOIN Prodotto AS PR ON (DETT.id_prodotto = PR.id_prodotto)
                        WHERE DETT.id_ordine = p_Id_Ordine;
RETURN CostoTotale;
```

```
END;
$$ LANGUAGE plpgsql;
```

AvailableCorrieri

Funzione con lo scopo di calcolare il numero di Corrieri disponibili per la spedizione.

```
CREATE OR REPLACE FUNCTION AvailableCorrieri()
RETURNS INTEGER
AS $$
DECLARE
N_Corrieri_Disp INTEGER;
BEGIN
                SELECT COUNT(*) INTO N_Corrieri_Disp
                FROM Corriere
                WHERE Disponibilita = TRUE;
RETURN N_Corrieri_Disp;
END;
$$ LANGUAGE plpgsql;
```

AvailableMezziDiTrasporto

Funzione con lo scopo di calcolare il numero di Mezzi di Trasporto disponibili per la spedizione.

```
CREATE OR REPLACE FUNCTION AvailableMezziDiTrasporto()
RETURNS INTEGER
AS $$
DECLARE
N_Mezzi_Disp INTEGER;
BEGIN
     SELECT COUNT(*) INTO N_Mezzi_Disp
     FROM Mezzo_di_trasporto
     WHERE Disponibilita = TRUE;
RETURN N_Mezzi_Disp;
END;
$$ LANGUAGE plpgsql;
```

checkamountpagamento

```
CREATE OR REPLACE FUNCTION public.checkamountpagamento(
        id_pagamento_param integer,
        importo_param double precision)
  RETURNS boolean
  LANGUAGE 'plpgsql'
AS $BODY$
BEGIN
    RETURN importo_param = (
        SELECT SUM(DETT.quantita * PR.prezzo)
        FROM Ordine AS O
        JOIN Dettaglio_Ordine AS DETT ON O.id_ordine = DETT.id_ordine
        JOIN Prodotto AS PR ON DETT.id_prodotto = PR.id_prodotto
        WHERE O.id_pagamento = id_pagamento_param
    );
END;
$BODY$;
```

checkcontraintarrivo

```
CREATE OR REPLACE FUNCTION public.checkamountpagamento(
        id_pagamento_param integer,
        importo_param double precision)
  RETURNS boolean
  LANGUAGE 'plpgsql'
AS $BODY$
BEGIN
    RETURN importo_param = (
        SELECT SUM(DETT.quantita * PR.prezzo)
        FROM Ordine AS O
        JOIN Dettaglio_Ordine AS DETT ON O.id_ordine = DETT.id_ordine
        JOIN Prodotto AS PR ON DETT.id_prodotto = PR.id_prodotto
        WHERE O.id_pagamento = id_pagamento_param
    );
END;
$BODY$;
```

md5password

```
CREATE OR REPLACE FUNCTION public.md5password(
        password_before character varying)
    RETURNS character varying
    LANGUAGE 'plpgsql'
AS $BODY$
BEGIN
        RETURN MD5(password_before);
END
$BODY$;
```

5.5 Procedure

PianificaSpedizione

Questa Procedura pianifica una nuova spedizione di un ordine specifico. Innanzitutto viene contata la quantità per Prodotto scelto dal cliente; viene poi confrontata con la quantità disponibile di quel Prodotto. Se il confronto va a buon fine, viene inserita la nuova spedizione dei Prodotti. Ciò viene fatto associando un Mezzo di Trasporto, scelto in base alla sua capacità e disponibilità, e un Corriere. La spedizione perciò viene aggiornata e il suo stato viene posto su 'spedito'.

```
CREATE OR REPLACE PROCEDURE PianificaSpedizione(id_ordine IN INTEGER)
LANGUAGE plpgsql
AS $$
DECLARE
quantita_ordine INTEGER;
id_spedizione INTEGER;
BEGIN
                        SELECT SUM(DETT.quantita) INTO quantita_ordine
                        FROM Dettaglio_Ordine AS DETT
                        WHERE DETT.id_ordine = id_ordine;

                        IF quantita_ordine > (
                                 SELECT SUM(P.quantita_disp)
                                 FROM Prodotto AS P
                                 JOIN Dettaglio_Ordine AS DETT
                                 ON (P.id_prodotto,DETT.id_prodotto)
                                 WHERE DETT.id_ordine = id_ordine) THEN
                        RAISE EXCEPTION 'La quantità richiesta supera la quantità disponibile.';
                        ELSE
                                        INSERT INTO Spedizione(destinazione, data_sped, società,
                                        stato, arrivo_prev, id_ordine)
                                        VALUES ('Indirizzo cliente', NULL, 'Società di spedizione',
                                        'ordinato', NULL, id_ordine);

      UPDATE Mezzo_di_Trasporto
      SET disponibilita = FALSE
      WHERE targa = (
            SELECT targa
            FROM Mezzo_di_Trasporto
            WHERE capacita_peso >= quantita_ordine
            AND disponibilita = TRUE
        );

      UPDATE Corriere
      SET disponibilita = FALSE
      WHERE n_telefono_corriere = (
            SELECT n_telefono_corriere
            FROM Corriere
            WHERE disponibilita = TRUE
        );

      UPDATE Spedizione
      SET stato = 'spedito', data_sped = CURRENT_DATE
      WHERE id_spedizione = id_spedizione;
    END IF;
END;$$
```

GestionePagamento

Questa Procedura gestisce il pagamento dell'ordine di un Cliente, verificando che questo sia stato effettuato con Esito 'TRUE'. Successivamente lo stato dell'Ordine corrispondente verrà modificato in quanto sarà andato a buon fine.

```
CREATE OR REPLACE PROCEDURE GestionePagamento(id_pagamento_param IN INTEGER)
LANGUAGE plpgsql
AS $$
DECLARE
importo_pagato DOUBLE PRECISION;
p_id_ordine INTEGER;
BEGIN
                SELECT importo, id_ordine INTO importo_pagato, p_id_ordine
                FROM Pagamento AS P
                WHERE id_pagamento_param = P.id_pagamento;

                IF importo_pagato = (
                        SELECT SUM(DETT.quantita * PR.prezzo)
                        FROM Ordine AS O
                        JOIN Dettaglio_Ordine AS DETT ON (O.id_ordine,DETT.id_ordine)
                        JOIN Prodotto AS PR ON (DETT.id_prodotto,PR.id_prodotto)
                        WHERE O.id_ordine = p_id_ordine) THEN

                        UPDATE Pagamento
                        SET esito = TRUE
                        WHERE id_pagamento = id_pagamento;

                        UPDATE Ordine
                        SET completamento = TRUE
                        WHERE id_ordine = p_id_ordine;

                        RAISE NOTICE 'Pagamento andato a buon fine, ordine confermato.';
                        ELSE
                                UPDATE Pagamento
                                SET esito = FALSE
                                WHERE id_pagamento = id_pagamento;
                        RAISE NOTICE 'Pagamento non riuscito.';
                END IF;

EXCEPTION
        WHEN NO_DATA_FOUND THEN
                RAISE NOTICE 'Non è stato trovato nessun pagamento.';
        WHEN OTHERS THEN
                RAISE NOTICE 'Errore nel pagamento dell''ordine.';

END; $$
```

StatoSpedizione

Questa Procedura viene attivata manualmente per modificare lo stato di una spedizione da 'spedito' a 'consegnato'.

```
CREATE OR REPLACE PROCEDURE StatoSpedizione(s_id_spedizione IN INTEGER)
LANGUAGE plpgsql
AS $$
DECLARE
s_stato_spedizione VARCHAR(20);
BEGIN
      SELECT stato
      FROM Spedzione
      WHERE id_spedizione = s_id_spedizione;

      IF s_stato_spedizione = 'spedito' THEN
         UPDATE Spedizione
         SET stato = 'consegnato'
         WHERE id_spedizione = s_id_spedizione;
                 RAISE NOTICE 'Stato della spedizione % aggiornato a consegnato.', s_id_spedizione;
          ELSE
         RAISE NOTICE 'La spedizione % non è ancora in consegna.', s_id_spedizione;
      END IF;

EXCEPTION
      WHEN NO_DATA_FOUND THEN
         RAISE NOTICE 'Spedizione non trovata.';
      WHEN OTHERS THEN
         RAISE NOTICE 'Errore durante la conferma di consegna della spedizione.';
   END; $$
```

AggiungiProdotto

Procedura che permette ad un operatore di aggiungere un nuovo prodotto, tramite la creazione di un nuovo ID.

```
CREATE OR REPLACE PROCEDURE AggiungiProdotto
(p_id_magazzino IN INTEGER, p_nome_prodotto IN VARCHAR(300), p_dimensioni IN DOUBLE PRECISION, p_peso
IN DOUBLE PRECISION,
p_quantita_disp IN INT, p_prezzo IN DOUBLE PRECISION)
LANGUAGE plpgsql
AS $$
DECLARE
nuovo_id_prodotto INTEGER;
BEGIN
                SELECT NVL(MAX(id_prodotto), 0) + 1
                INTO nuovo_id_prodotto
                FROM Prodotto;

                INSERT INTO Prodotto(id_prodotto, nome_prodotto, dimensioni, peso, quantita_disp,
                prezzo, id_magazzino)
                VALUES (nuovo_id_prodotto, p_nome_prodotto, p_dimensioni, p_peso, p_quantita_disp,
                p_prezzo, p_id_magazzino);

```

```
                RAISE NOTICE 'È stato inserito un nuovo prodotto: % ', nuovo_id_prodotto;

EXCEPTION
        WHEN OTHERS THEN
        RAISE NOTICE 'Si è verificato un errore durante l''aggiunta del prodotto.';
END;$$
```

AggiornaQuantitaProdotto

Procedura che ha il compito di modificare la quantità di prodotti disponibili aventi un determinato ID.

```
CREATE OR REPLACE PROCEDURE AggiornaQuantitaProdotto (p_id_prodotto IN INT, nuova_quantita IN INT)
LANGUAGE plpgsql
AS $$
DECLARE
BEGIN
                UPDATE Prodotto
                SET quantita_disp = nuova_quantita
                WHERE id_prodotto = p_id_prodotto;

                RAISE NOTICE 'Quantità del prodotto % modificata.', p_id_prodotto;

EXCEPTION
        WHEN NO_DATA_FOUND THEN
                RAISE NOTICE 'Il prodotto non è stato trovato.';
        WHEN OTHERS THEN
                RAISE NOTICE 'Errore durante la modifica della quantità del prodotto.';
END;$$
```

6 Implementazione in Java della classe DAO

6.1 Entità ordine

```java

import java.time.LocalDate;

public class Ordine {
    private int idOrdine;
    private LocalDate dataOrdine;
    private boolean completamento;
    private String emailCliente;
    private int numeroProdotti;

    public Ordine(int idOrdine, LocalDate dataOrdine, Boolean completamento, String emailCliente, int numeroProdotti) {
        this.idOrdine = idOrdine;
        this.dataOrdine = dataOrdine;
        this.completamento = completamento;
        this.emailCliente = emailCliente;
        this.numeroProdotti = numeroProdotti;
    }

    public String getEmailCliente() {
        return emailCliente;
    }

    public int getIdOrdine() {
        return idOrdine;
    }

    public LocalDate getDataOrdine() {
        return dataOrdine;
    }

    public int getNumeroProdotti() {
        return numeroProdotti;
    }

    @Override
    public String toString() {
        return "Ordine{" +
```

```
                "idOrdine=" + idOrdine +
                ", dataOrdine=" + dataOrdine +
                ", completamento=" + completamento +
                ", emailCliente='" + emailCliente + '\'' +
                ", numeroProdotti=" + numeroProdotti +
                '}';
    }
}

```

6.2 Ordine Dao

```

import com.uninadelivery.model.dbconnection.DBConnection;
import com.uninadelivery.model.entities.Ordine;

public class OrdineDAO {
    private static final Logger LOGGER = Logger.getLogger(OrdineDAO.class.getName());

    public List<Ordine> getAllOrdini() {
        List<Ordine> orders = new ArrayList<>();
        String query = "SELECT o.id_ordine, o.data_ordine, o.completamento, o.email_cliente, " +
                "COALESCE(SUM(d.quantita), 0) AS numero_prodotti " +
                "FROM ordine o " +
                "LEFT JOIN dettaglio_ordine d ON o.id_ordine = d.id_ordine " +
                "GROUP BY o.id_ordine, o.data_ordine, o.completamento, o.email_cliente";

        try (Connection conn = DBConnection.getDBconnection().getConnection();
             PreparedStatement ps = conn.prepareStatement(query);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                int idOrdine = rs.getInt("id_ordine");
                LocalDate dataOrdine = rs.getDate("data_ordine").toLocalDate();
                Boolean completamento = rs.getBoolean("completamento");
                String emailCliente = rs.getString("email_cliente");
                int numeroProdotti = rs.getInt("numero_prodotti");

                Ordine ordine = new Ordine(idOrdine, dataOrdine, completamento, emailCliente,
                numeroProdotti);
                orders.add(ordine);
            }
        } catch (SQLException e) {
            LOGGER.log(Level.SEVERE, "Errore durante il recupero degli ordini", e);
        }

        return orders;
    }

    public List<Integer> getOrdiniNonAssegnati() {
        List<Integer> ordiniNonAssegnati = new ArrayList<>();
        String query = "SELECT id_ordine FROM ordine " +
```

```
                "WHERE id_ordine NOT IN (SELECT id_ordine FROM spedizione)";

        try (Connection conn = DBConnection.getDBconnection().getConnection();
             PreparedStatement ps = conn.prepareStatement(query);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                ordiniNonAssegnati.add(rs.getInt("id_ordine"));
            }

        } catch (SQLException e) {
            LOGGER.log(Level.SEVERE, "Errore nel recupero degli ordini non assegnati", e);
        }
        return ordiniNonAssegnati;
    }

}
```